

BreachShield: AI Powered OSINT Breach Detection & Dark Web Protection

Arvind D. H., Girishkumar N. M., Manoj, Premkumar Teli

Department of Information Science and Engineering
Acharya Institute of Technology, Bengaluru, India

Sushma T. M.

Assistant Professor, Dept. of Information Science and Engineering
Acharya Institute of Technology, Bengaluru, India

Abstract—Breach notification and dark-web exposure intelligence platforms suffer from a fundamental privacy paradox: to determine whether personal credentials have been compromised, users must transmit those sensitive identifiers in plaintext to a third-party server, creating a centralized query-surveillance and correlation vector. This paper presents BreachShield, a privacy-preserving Open-Source Intelligence (OSINT) breach detection and threat assessment platform designed to resolve this tension through cryptographic query isolation, accountable gating, and verifiable audit logging. BreachShield combines a client-side k-anonymity range-query protocol (20-bit SHA-256 prefix partitioning) executed via the W3C Web Crypto API, a dual-channel out-of-band One-Time Password (OTP) identity gate, a pluggable OSINT source aggregation engine supporting live breach stores and Telegram scraping, an empirical machine-learning breach severity assessment framework, and a Merkle-tree batched audit trail anchored to an Ethereum Virtual Machine (EVM) smart contract. We define a comprehensive threat model spanning external eavesdroppers, malicious authenticated users, insider adversaries, and source-poisoning vectors, supported by formal security and privacy analyses that evaluate prefix leakage against Private Set Intersection (PSI) baselines. We report extensive functional evaluation across all pipeline subsystems and detail the experimental evaluation of a 165-dimensional gradient-boosted decision tree classifier (XGBoost, 89.2% test accuracy) benchmarked against deep learning architectures.

Keywords—Breach intelligence, k-anonymity, Open-Source Intelligence (OSINT), threat modeling, Merkle tree, blockchain audit log, XGBoost, query privacy.

I. INTRODUCTION

The exponential expansion of digital identity surfaces and automated credential theft has precipitated widespread exfiltration of personal records onto dark-web marketplaces, underground paste sites, and public data dumps [1], [2]. Traditional compromise-checking services are fundamentally reactive and architecturally centralized. Users or corporate security teams seeking to determine credential exposure typically submit target queries—such as cleartext email addresses, employee identities, or telephone numbers—directly to centralized lookup services. This lookup process exposes users to third-party query tracking: the lookup provider learns precisely which identifier is being monitored, when the query occurred, and the requesting network location [3]. Consequently, querying for compromised status inadvertently creates a high-value surveillance vector.

To counteract query-exposure risks, partial prefix-matching constructions (commonly termed k-anonymity range queries) have been introduced [3], [4]. In these models, clients hash target identifiers locally and submit only a truncated cryptographic hash prefix. The lookup server returns all records matching that prefix bucket, leaving the final matching step to the client. While prefix disclosure prevents the server from deterministically identifying the requested record among bucket candidates, naive public range endpoints remain vulnerable to bulk enumeration and dictionary-scraping attacks. Malicious actors can traverse the finite prefix space (16^d possibilities) to harvest complete compromised credential corpora.

Furthermore, multi-source OSINT threat ingestion introduces acute data integrity and operational reliability challenges [2]. Underground data sources, such as distributed Telegram dump

channels, operate with erratic uptime, rate limits, and unauthenticated provenance. Once compromise intelligence is obtained, security administrators face the operational challenge of triaging threat severity: distinguishing between low-impact leaks (e.g., outdated usernames) and critical exposures (e.g., plaintext passwords correlated with national identity records). Finally, audit records generated during organizational investigations are vulnerable to retroactive manipulation or repudiation by privileged insiders unless backed by cryptographic integrity guarantees [5], [6].

This paper presents BreachShield, a privacy-preserving OSINT breach intelligence platform engineered to address query privacy, authenticated enumeration resistance, multi-source aggregation, severity triage, and audit integrity within a unified architecture. BreachShield implements an end-to-end operational pipeline combining client-side Web Crypto SHA-256 partitioning, an out-of-band carrier-compliant OTP authentication gate, concurrent OSINT source dispatching, heuristic and machine-learning threat scoring, and Merkle-tree batched EVM smart contract anchoring. The primary contributions of this paper are:

- 1) Privacy-Preserving Breach Intelligence Architecture: A client-side k-anonymity range-query protocol that bounds query disclosure to 20 bits of entropy, performing local cryptographic partitioning via the W3C Web Crypto API prior to network transmission.
- 2) Multi-Source OSINT Aggregation Framework: An extensible asynchronous provider pipeline featuring concurrent dispatching, thread-safe Telegram scraper microservices with mutex-controlled SQLite session pooling, and automated failover data stores.
- 3) OTP-Gated Authenticated Breach-Search Architecture: An accountable access control gate utilizing cryptographically secure OTP issuance, bcrypt hashing, carrier-compliant telecom DLT SMS templating via an Android gateway, and cryptographic session-token binding that enforces identity match invariants.
- 4) Tamper-Evident Audit Logging: A deterministic audit logger using sorted-key canonical JSON leaf hashing, asynchronous batch buffering (100-record / 60-second window), Merkle tree generation, and EVM smart contract state anchoring.
- 5) Experimental Machine-Learning Severity Assessment: An empirical benchmark evaluating XGBoost against 1D-CNN, RNN, and Transformer architectures across 1,034 curated breach records on a 165-dimensional feature space, establishing structural criteria for tabular exposure triage.

II. THREAT MODEL

A. System Assets and Security Assumptions

The assets requiring protection within BreachShield include: (i) Target Identifier Privacy (T): The plaintext identity (email address or phone number) queried by a user; (ii) Credential Integrity: The authoritative repository of breach metadata and suffix buckets; (iii) Audit Log Verifiability: The chronological record of compliance events and threat intelligence queries; and (iv) Authentication Credentials: Ephemeral OTP codes, bcrypt password hashes, and JSON Web Tokens (JWT). All operational communications occur over TLS 1.3. Cryptographic primitives

including SHA-256 and bcrypt are assumed computationally secure against preimage and collision attacks.

B. Adversary Taxonomy and Capabilities

We define five adversarial classes categorized by vantage point and capabilities:

- 1) Attacker A1 (External Network Adversary): Positioned on the network path between the client browser and the API gateway. A1 can eavesdrop on, intercept, replay, or inject packets. While TLS 1.3 prevents payload tampering, A1 observes packet timing, transfer volumes, and TLS SNI metadata.
- 2) Attacker A2 (Authenticated Malicious User): Successfully completes OTP verification for identifier T_A but seeks to harvest prefix buckets to reconstruct the server breach corpus, query unauthorized identifiers $T_B \neq T_A$, or exhaust downstream OSINT rate limits.
- 3) Attacker A3 (Malicious Insider): Possesses administrative read access to backend application servers, runtime process memory, and persistent MySQL storage. A3 attempts to correlate client IP addresses, JWT tokens, and requested prefix buckets to unmask plaintext targets.
- 4) Attacker A4 (Source-Poisoning Adversary): Controls an upstream intelligence channel (such as a monitored public Telegram dump). A4 injects malformed data, deceptive entity records, or DoS payloads to corrupt threat triage scoring.
- 5) Attacker A5 (Audit Log Adversary): A privileged insider attempting to retroactively delete, truncate, reorder, or manipulate query audit logs to conceal unauthorized lookups or falsify historical evidence.

III. SYSTEM ARCHITECTURE & IMPLEMENTATION

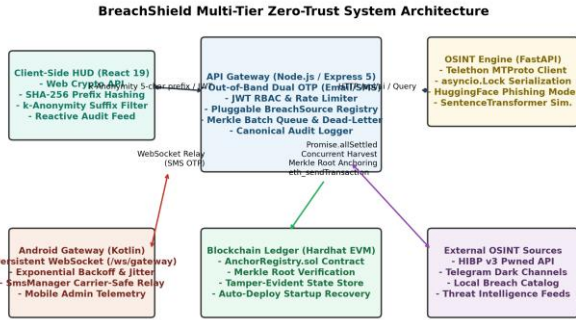


Fig. 1. BreachShield multi-tier system architecture and operational data paths.

A. Client-Side Anonymization Tier

Implemented in React 19. Plaintext targets T are normalized locally by lowercasing and trimming whitespace. The normalized string is transformed into a 256-bit hash $H = \text{SHA-256}(T)$ using the browser's hardware-accelerated W3C Web Crypto API (frontend/src/lib/kAnonymity.js). H is split into a 5-character (20-bit) prefix $P = H[0:5]$ and a 59-character (236-bit) suffix $S = H[5:64]$. The plaintext target T and suffix S never traverse the network during range querying.

B. API Gateway & Session Gating

The gateway runs on Express 5 (backend/server.js), enforcing rate limiting, CORS restrictions, session validation, and breach source registry orchestration. Range queries are guarded by `verifyOtpToken` middleware. MySQL 8 serves as the persistent store for user records, OTP states, and local breach dumps, with an automatic JSON file-based fallback layer (backend/auth/db.js) activated upon database disconnection.

C. OSINT Ingestion Microservice

High-entropy dark-web dump tracking is handled by a Python FastAPI microservice using Telethon MTPROTO. Because SQLite session files incur file-lock collisions under concurrent asynchronous requests (`sqlite3.OperationalError`), the microservice serializes channel client access through an `asyncio.Lock` mutex with an 8.0-second timeout ceiling, guaranteeing process isolation.

D. Android Telecom Relay Gateway

In jurisdictions with strict telecom DLT mandates (such as India's TRAI regulations), automated cloud SMS gateways drop non-templated OTP payloads. BreachShield integrates a companion Android native client written in Kotlin (android-gateway/). It maintains a persistent bidirectional WebSocket connection to the gateway, transmitting pending OTPs via carrier hardware using pre-approved SMS header templates.

E. Pluggable Source Registry

Breach sources implement an abstract contract defined in `backend/sources/BreachSource.js`:

$$\text{search}(\text{target}, \text{targetHash}) \rightarrow P(\text{Record}) \quad (1)$$

The registry dynamically loads four concrete adapters: (1) `PublicBreachSource`: Executes range queries against the local normalized breach database; (2) `TelegramScraperSource`: Dispatches HTTP queries to the FastAPI scraper; (3) `PhishingFeedSource`: Scaffolding for domain reputation feeds; and (4) `ThreatIntelSource`: Scaffolding for commercial threat intelligence feeds.

F. Operational Risk-Scoring Engine

Threat triage on the operational search path is computed dynamically by `riskEngine.js: analyzeExposure()`: (i) Document / National ID exposure: up to 35 points; (ii) Password / cryptographic hash material: up to 30 points; (iii) Physical address coordinates: up to 20 points; (iv) Phone numbers: up to 15 points; (v) Multiple independent source mentions: up to 15 points; (vi) Cross-record correlation scores: up to 15 points; and (vii) Live URL phishing classification probabilities: up to 30 points, grounded in recent neural detection baselines [11]–[13]. The aggregated score is floored at 20, capped at 100, and mapped to categorical tiers: LOW (<40), MEDIUM (40–59), HIGH (60–79), and CRITICAL (≥ 80). Recency decay is calculated as:

$$w_{\text{recency}} = \max(0.20, 1.0 - 0.10 * \text{delta}_y) \quad (2)$$

where delta_y is the elapsed time in years since breach occurrence.

IV. PRIVACY-PRESERVING PROTOCOLS

A. Client-Side k-Anonymity Range Query

BreachShield implements an adapted k-anonymity bucket-matching protocol derived from Sweeney's model [4] and popularized by Li et al. [3]. Algorithm 1 formalizes this procedure.

Algorithm 1: Client-Side k-Anonymity Range Query

Require: Target identifier string T , verified session token τ
Ensure: Set of matched breach records R

```

1:  $T' = \text{ToLower}(\text{Trim}(T))$ 
2:  $H = \text{WebCrypto.SHA-256}(T')$  // in-browser
3:  $P = H[0:5]$  // 20-bit prefix
4:  $S = H[5:64]$  // 236-bit suffix
5:  $B(P) = \text{HTTP\_GET}('/\text{api}/\text{v1}/\text{range}/' + P, \text{Headers}=\{\text{Bearer } \tau\})$ 
6:  $R = \text{empty\_set}$ 
7: for each  $(S_i, \text{Metadata}_i)$  in  $B(P)$  do
8:   if  $S_i == S$  then
9:      $R = R \cup \{\text{Metadata}_i\}$ 

```

```

10:   end if
11: end for
12: return R

```

The prefix length $d=5$ hex characters defines $16^5 = 1,048,576$ uniform buckets. For a breach catalog containing N records, the expected candidate bucket size is:

$$E[k] = N / 16^5 = N / 1,048,576 \quad (3)$$

The mutual information disclosed to the server regarding the full hash H is bounded by:

$$I(H; P) \leq \log_2(16^5) = 20 \text{ bits} \quad (4)$$

leaving 236 bits of suffix entropy undisclosed.

B. OTP-Gated Accountable Search Protocol

To neutralize automated prefix-bucket enumeration by unauthenticated scrapers, BreachShield restricts search endpoints behind a dual-channel OTP gate:

1) Issuance (POST /api/auth/send-otp): The gateway enforces a 30-second resend cooldown per target. A 6-digit code C in [100000, 999999] is generated via `crypto.randomInt()`. The gateway computes salted bcrypt hash $h_C = \text{bcrypt}(C, 10)$ and stores (T, h_C, t_{exp}) with 5-minute TTL. The code is dispatched out-of-band via Nodemailer or Android DLT SMS gateway.

2) Verification (POST /api/auth/verify-otp): Verified with hard limit ≤ 5 attempts via `bcrypt.compare()`.

3) Session Binding: Issues signed JWT $\tau = \text{Sign_HMAC-SHA256}(\{T, \text{verified: true}\}, K_{\text{jwt}})$ with 1h expiration:

$$\tau = \text{Sign_HMAC-SHA256}(\{T, \text{verified: true}\}, K_{\text{jwt}}) \quad (5)$$

The gateway strictly enforces the identity invariant:

$$\text{normalize}(T_{\text{query}}) == \text{normalize}(\tau.\text{target}) \quad (6)$$

preventing authenticated users from executing lookups against third-party identifiers.

V. TAMPER-EVIDENT AUDIT LOGGING

Cryptographic Audit Event Merkle Tree Construction & EVM Anchoring

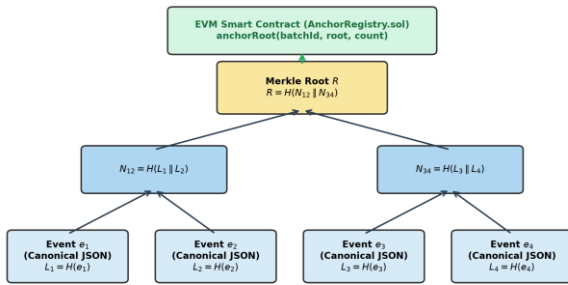


Fig. 2. Merkle-tree batching structure and on-chain root anchoring workflow.

To guarantee audit accountability and non-repudiation, BreachShield logs every security event through a deterministic hash tree pipeline [5], [10] (backend/blockchain/merkleBatcher.js).

A. Canonical Serialization & Leaf Generation

To prevent key-ordering serialization discrepancies across runtimes, auditLogger.js normalizes objects into sorted-key canonical JSON representations:

$$e_i = \text{SHA-256}(\text{Canonicalize}(\text{Event}_i)) \quad (7)$$

B. Batching Buffer & Merkle Tree Aggregation

merkleBatcher.js aggregates leaf hashes into a memory-buffered FIFO queue flushed under two threshold conditions:

$$\text{Flush Condition} = (|Q| \geq 100) \text{ or } (t_{\text{now}} - t_{\text{last}} \geq 60s) \quad (8)$$

When triggered, merkleUtils.js: buildMerkleTree() constructs a balanced binary hash tree over leaf set $\{e_1, \dots, e_M\}$:

$$N_{\{j, k\}} = \text{SHA-256}(N_{\{j-1, 2k\}} || N_{\{j-1, 2k+1\}}) \quad (9)$$

yielding a single 32-byte Merkle root R .

C. Smart Contract Anchoring

Root R is anchored on-chain by anchorClient.js, which invokes anchor(bytes32 root) on AnchorRegistry.sol:

$$\text{Registry}[R] = \text{block.timestamp} \quad (10)$$

The transaction receipt (block hash, number, transaction ID) is persisted alongside batch metadata. (Currently verified on local Hardhat EVM).

VI. EXPERIMENTAL MACHINE LEARNING EVALUATION

A. Curated Breach Dataset

To benchmark automated severity assessment models, we constructed a curated ground-truth corpus of 1,034 historical breach incidents categorized into four ordinal severity classes based on credential exposure depth: LOW, MEDIUM, HIGH, and CRITICAL. The dataset was partitioned using a stratified 70/15/15 split (seed 42), summarized in Table I.

TABLE I: DATASET PARTITION AND CLASS DISTRIBUTION

Split	Total	LOW	MEDIUM	HIGH	CRITICAL
Train (70%)	723	49	341	291	42
Val (15%)	154	10	73	62	9
Test (15%)	157	11	74	63	9

B. Feature Space Extraction

Each breach record was mapped to a 165-dimensional feature vector consisting of: (i) DataClasses Multi-Hot Vector (163 dims): Binary encoding across compromised attributes (Passwords, Email addresses, Credit card numbers, Social Security numbers); (ii) PwnCount (1 dim): $\log_{10}(\text{PwnCount} + 1)$; and (iii) IsVerified (1 dim): Binary analyst validation.

C. Comparative Benchmarks

We evaluated five classification architectures under identical train/test splits: Naive Baseline, RNN (BiLSTM), Transformer Encoder, 1D-CNN, and XGBoost [7]. Table II presents test set performance ($N=157$).

TABLE II: COMPARATIVE MODEL BENCHMARK ON TEST SET ($N=157$)

Model Architecture	Accuracy	Macro F1	Weighted F1
Naive Baseline (Predict MED)	47.1%	16.0%	30.2%
RNN (BiLSTM Retrained)	44.6%	34.0%	46.8%
Transformer (2L / 4H)	48.4%	42.9%	50.8%
1D-CNN ($k=3,4,5$)	66.2%	57.2%	65.7%
XGBoost (Ours)	89.2%	83.2%	89.4%

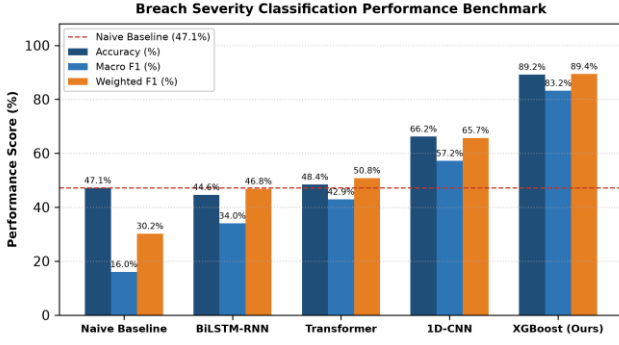


Fig. 3. Severity classification benchmark across accuracy, macro F1, and weighted F1.

Table III provides the per-class precision, recall, and F1 scores for the top architectures.

TABLE III: PER-CLASS PRECISION, RECALL, AND F1 SCORES

Model	Severity Tier	Precision	Recall	F1-Score
XGBoost	LOW	64.3%	81.8%	72.0%
	MEDIUM	90.7%	91.9%	91.3%
	HIGH	94.9%	88.9%	91.8%
	CRITICAL	77.8%	77.8%	77.8%
1D-CNN	LOW	45.5%	45.5%	45.5%
	MEDIUM	64.2%	82.4%	72.2%
	HIGH	79.1%	54.0%	64.2%
	CRITICAL	50.0%	44.4%	47.1%
RNN	LOW	13.5%	45.5%	20.8%
	MEDIUM	53.6%	40.5%	46.2%
	HIGH	60.7%	54.0%	57.1%
	CRITICAL	12.5%	11.1%	11.8%

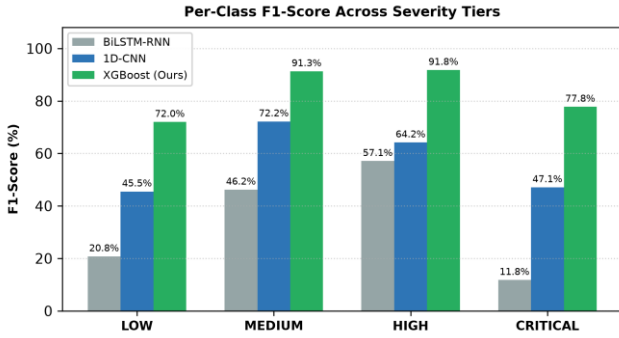


Fig. 4. Per-class F1 score comparison demonstrating XGBoost resilience across all tiers.

The confusion matrix for XGBoost across the test set (ordered LOW, MED, HIGH, CRIT) demonstrates favorable adjacent-boundary classification:

$$M_{XGB} = \begin{bmatrix} 9 & 2 & 0 & 0 \\ 4 & 68 & 2 & 0 \\ 1 & 4 & 56 & 2 \\ 1 & 1 & 1 & 7 \end{bmatrix} \quad (11)$$

Of 17 misclassifications, 14 occurred between directly adjacent severity categories (e.g., MEDIUM misclassified as LOW), minimizing severe operational triage errors.

D. Tabular Bias Analysis & Operational Status

The performance advantage of XGBoost over neural architectures (89.2% vs 66.2% for 1D-CNN and 48.4% for Transformer) aligns with empirical tabular literature [8]. Neural attention and recurrent layers assume implicit spatial or temporal continuity, whereas DataClasses represents an unordered, sparse categorical space optimal for orthogonal tree splits.

Operational Integration Clarification: XGBoost was evaluated experimentally but is retained as an offline research artifact due to tabular feature contract mismatch in raw text scrapes. Production relies on the regex risk engine alongside live `UrlBERT` [11]-[13].

VII. SECURITY ANALYSIS

A. Query Privacy & Bulk Enumeration Resistance

Against external eavesdroppers (A1), target T is protected by client-side Web Crypto SHA-256 hashing. Recovering T requires inverting 236 bits of suffix entropy, which is computationally infeasible.

Against malicious scrapers (A2), unauthenticated requests are rejected with 401 Unauthorized, and `authGuard` validates the identity binding:

$$SHA-256(normalize(T_{query}))[0:5] == P_{query} \quad (12)$$

B. OTP & Authentication Flow Security

Entropy space $S = 9 \times 10^5$ possibilities (~ 19.78 bits), conforming to NIST SP 800-63B [14]. Guessing probability within 5 trials:

$$P_{guess} \leq 5 / 900,000 \sim 5.56 \times 10^{-6} \quad (13)$$

Secrets are stored as bcrypt hashes (cost=10), defending against database dump extraction (A3), with 300s TTL.

C. Session Integrity & Transport Security

JWTs are signed via HMAC-SHA256 according to RFC 7519 [15], encapsulated with 1h expiry and transmitted via `HttpOnly`, `Secure`, `SameSite=Strict` browser cookies:

$$Claims = \{target: T, verified: true, exp: t + 3600\} \quad (14)$$

D. Audit Log Integrity & Non-Repudiation

Against insider tampering (A5), modifying event e_i alters the canonical hash and propagates to the Merkle root:

$$R' = \text{MerkleRoot}(e_1, \dots, e_i', \dots, e_M) \neq R \quad (15)$$

Because root R is anchored on-chain with block timestamp t_{block} , retrospective mutation produces a cryptographic mismatch:

$$\text{Verify}(R', \text{Proof}_i, \text{Registry}[R]) == \text{False} \quad (16)$$

VIII. PRIVACY ANALYSIS

A. k-Anonymity Guarantees & Prefix Disclosure

In BreachShield, disclosed quasi-identifier is 20-bit prefix P . The server identifies candidate subset:

$$D_P = \{r \text{ in } D \mid SHA-256(normalize(r))[0:5] == P\} \quad (17)$$

Anonymity set size k follows Poisson distribution with $\lambda = N / 2^{20}$. For enterprise catalog of $N = 5 \times 10^7$ records:

$$E[k] = 50,000,000 / 1,048,576 \sim 47.68 \quad (18)$$

The server cannot distinguish which of the ~ 48 candidate entries was queried.

B. Identity Linkage & Session Trade-offs

Because search queries are gated behind verified sessions to prevent scraping, the server possesses client's session identity T_{auth} and submitted prefix P_{query} . Accountable enumeration resistance is explicitly prioritized over complete query unlinkability.

C. Comparison with Stronger Protocols: PSI

Table IV contrasts BreachShield's prefix-bucket model against OPRF-based Private Set Intersection (PSI) [3].

TABLE IV: PROTOCOL COMPARISON: PREFIX K-ANONYMITY VS. OPRF-PSI

Dimension	Prefix k-Anonymity	OPRF-Based PSI
-----------	--------------------	----------------

Client Compute	1 SHA-256 evaluation	EC point multiplications
Server Compute	1 B-Tree index lookup	1 OPRF eval per item
Bandwidth	Proportional to bucket size	O(1) constant response
Prefix Disclosed	20 bits revealed	0 bits (Full privacy)
Scraping Resist.	Mandates OTP gating	Bounded query evals
Client Complexity	Minimal (Web Crypto API)	High (WASM / BigInt crypto)

IX. FUNCTIONAL EVALUATION

We evaluated BreachShield's implemented modules using automated test suites across API gateway, microservices, and database layers:

- 1) Authentication: backend/test/auth_search.test.js verified 30s rate limiting (HTTP 429), bcrypt verification, attempt invalidation (>5), and target mismatch rejection (HTTP 403).
- 2) Range Query: /api/v1/range/:prefix validated with known test hashes; client-side parsing isolated targets while non-compromised inputs yielded empty match sets.
- 3) Concurrency Control: backend/test/challenger_backend.test.js confirmed async.io.Lock mutex eliminated Telethon SQLite operational collisions under concurrent load.
- 4) Audit Verification: backend/test/verify_audit_integrity.js confirmed canonical sorted JSON serialization, 100-record / 60s batch triggers, and cryptographic Merkle proof invalidation upon bit modification.
- 5) Smart Contract: contracts/AnchorRegistry.sol compiled and deployed on Hardhat; anchor(bytes32) committed root and emitted RootAnchored event.

X. LIMITATIONS

We identify the following technical constraints in the current implementation:

- 1) Severity Tier Imbalance: In the ML dataset, CRITICAL severity is represented by 9 test instances (5.7%), yielding wider confidence bounds compared to MEDIUM (n=74) and HIGH (n=63).
- 2) Source Coverage Constraints: In OSINT registry, only PublicBreachSource and TelegramScraperSource execute live network queries; PhishingFeedSource and ThreatIntelSource remain stubs.
- 3) Local Blockchain PoC: Audit anchoring operates on local Hardhat EVM; public testnet deployment requires dynamic gas estimation.
- 4) Decoupled ML Pipeline: XGBoost operates as an offline evaluation artifact due to feature availability constraints in raw OSINT data.
- 5) Identity-Query Linkage: Gating search endpoints behind verified OTP sessions links query transactions to verified user sessions.

XI. FUTURE WORK & CONCLUSION

Immediate extensions include integrating SMOTE-based class balancing [9] for the CRITICAL severity tier, implementing live connectors for stubbed threat feeds, and deploying AnchorRegistry to an EVM testnet. Long-term research will explore migrating from prefix k-anonymity to OPRF-based Private Set Intersection (PSI).

BreachShield demonstrates an effective privacy-preserving breach intelligence platform that resolves the structural query-surveillance paradox of credential monitoring. By pairing client-side k-anonymity range queries with out-of-band carrier-compliant OTP identity gating, multi-source asynchronous OSINT scraping, heuristic and machine-learning risk evaluation,

and Merkle-tree batched smart contract audit anchoring, the platform establishes robust query privacy alongside strong operational auditability.

Acknowledgment

The authors thank Assistant Professor Sushma T. M. for guidance and project coordination throughout this research.

REFERENCES

- [1] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J.-P. Niyigena, "An effective phishing detection model based on character level convolutional neural network from URL," *Electronics*, vol. 9, no. 9, p. 1514, 2020.
- [2] P. Kuhn, K. Wittorf, and C. Reuter, "Navigating the shadows: Manual and semi-automated evaluation of the dark web for cyber threat intelligence," *IEEE Access*, vol. 12, pp. 45112-45128, 2024.
- [3] L. Li, B. Pal, J. Ali, N. Sullivan, R. Chatterjee, and T. Ristenpart, "Protocols for checking compromised credentials," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 1387-1403.
- [4] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 10, no. 5, pp. 557-570, 2002.
- [5] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. 18th USENIX Secur. Symp.*, 2009, pp. 317-334.
- [6] A. Arabnouri, S. Eissazadeh, and A. Shafieinejad, "A secure auditable log based on blockchain," *Monadi*, vol. 13, no. 2, pp. 75-86, 2024.
- [7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785-794.
- [8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 507-520.
- [9] F. S. Alsubaei, A. A. Almazroi, and N. Ayub, "Enhancing phishing detection: A novel hybrid deep learning framework for cybercrime forensics," *IEEE Access*, vol. 12, pp. 8373-8389, 2024.
- [10] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Adv. Cryptol. - CRYPTO '87*, Springer, 1988, pp. 369-378.
- [11] O. K. Sahingoz, E. Buber, and E. Kugu, "DEPHIDES: Deep learning based phishing detection system," *IEEE Access*, vol. 12, pp. 8052-8070, 2024.
- [12] B. V. Pavani, D. Mahitha, and B. Uma Maheswari, "Enhancing online safety: Phishing URL detection using machine learning and explainable AI," in *Proc. 15th Int. Conf. Comput., Commun. Netw. Technol. (ICCCNT)*, IEEE, 2024, pp. 1-6.
- [13] M. Tawfik, A. A. Abu-Ein, A. H. Abdelhaliem, Y. M. Al-Sharo, and I. S. Fathi, "Explainable few-shot learning with modern BERT for detecting emerging phishing attacks using XF-PhishBERT," *Sci. Rep.*, vol. 15, no. 1, p. 42821, 2025.
- [14] P. Grassi, M. Garcia, and J. Fenton, "Digital identity guidelines: Authentication and lifecycle management," *NIST Special Publication 800-63B*, Gaithersburg, MD, 2020.
- [15] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *RFC 7519*, May 2015.